Lab 01 - Code-First Setup

Scenario

Throughout the labs you'll work on a console app called 'School System'. The UI of this app has been built by another team already. It consists of menus and submenus. Also, a couple of services have been created, but they don't do anything yet, apart from showing a message in the console.

You will be responsible for applying *all* logic. From building and filling a new `School` database, up to adding logic to query and modify data in all kinds of ways. Good luck!

Exercise 1: DbContext

1. Open the provided solution `School` and run the project `School.ConsoleApp`. Test some menus.
2. Stop the app and investigate the menu logic and services.
3. Add the NuGet package

   `Microsoft.EntityFrameworkCore.SqlServer`.
4. Add a 'DAL' folder (*DAL* means *Data Access Layer*) to the project. Inside add a `SchoolContext` class that inherits from `DbContext`.
5. Add a constructor that accepts a parameter `options` of type `DbContextOptions<SchoolContext>`, pass `options` to the base. Keep the body of the constructor empty.
6. Configure the options in the `Main` method of the `Program` class, just before the services are instantiated.
   The name of the database should be **`School2`**:

   ```
   var options = new DbContextOptionsBuilder<SchoolContext>()
      .UseSqlServer(@"Connection string here")
      .Options;
   ```
7. Ensure that the database is created afterwards:

   ```
   using var context = new SchoolContext(options);
   // Add line here to create the database
   ```
8. Compile and run the application. Check if a database **`School2`** has been created.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 2: Model

1. Add a 'Models' folder. Inside add a new `Person` class. Add the properties `Id`, `FirstName` and `LastName`. Example:

```
public string FirstName { get; set; } = null!;
```

2. Add a property to the `DbContext` and map the `Person` class to the `People` table.
3. Compile and run the application. The database should have been recreated. Check if the table `People` is available.

»

Exercise 3: Add data

1. In the 'DAL' folder add a new `SchoolInitializer` class with one static `Seed` method. This method should expect an `options` parameter of type `DbContextOptions<SchoolContext>`.
2. As done before, open a new instance of `SchoolContext` and pass `options`. Don't forget the `using` keyword.
3. Copy the code from `Reference/seed-lab01.txt` and paste it into the `Seed` method.

   Later you will learn more about seeding a database.
4. The database already contains an empty table. To ensure the old database is deleted *before* a new database is created, add a line of code in the `Program` class.
5. Call the static `Seed` method after creating the database.
6. Compile and run the application. There should be no errors. Show all people in the next exercise.

»

Exercise 4: Read & display data

1. Displaying people will happen in the `PersonService`. Add logic to this service so that the `DbContextOptions<SchoolContext>` can be injected:

```
private readonly DbContextOptions<SchoolContext> _options;

public PersonService(DbContextOptions<SchoolContext> options)
{
    _options = options;
}
```

2. When instantiating the `PersonService`, pass `options`:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
var personService = new PersonService(options);
```

3. Add logic to the `DisplayAllPeople` method so that all people are displayed this way:

```
All People
 1 - Fadi Fakhouri
 2 - Stacy Serrano
 3 - Jasmine Stewart
 ...
```

4. Test.
5. Repeat step 1 and 2 for the other services as well to make them database-ready.

»

Exercise 5: Use configuration file

(if time permits)

1. Move the connection string to a separate `appsettings.json` file and verify the application still works.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 02 - Entity Configuration

Scenario

You'll create two new tables, configure them using both data annotations and fluent API, and load them up with sample data to display.
Finally refactor the configuration into separate classes for each entity.

Exercise 1: Configure Person model - Data Annotations

1. Check in the database the types of the columns in the `People` table. In SQL Server the two text columns are of type `nvarchar(MAX)`. This datatype is meant for large amounts of text. In this case we want to use a datatype that is fit for small amounts of text.
2. Update the table columns by using data annotations in the `Person` model:
   - **`Id`** is a primary key, autogenerated
   - **`FirstName`** has maximum length 50, not nullable
   - **`LastName`** has maximum length 50, not nullable
3. Compile and run the application. Check the fields in the database again. They should be OK now.

Exercise 2: Add two models

1. Ensure next two tables are built and configured using Fluent API:
   - Courses
     - **`Id`**, `int`, primary key, **not generated by the database**
     - **`Title`**, maximum length 100, not nullable
     - **`Credits`**, `int`, not nullable
   - Departments
     - **`Id`**, `int`, primary key, autogenerated
     - **`Name`**, maximum length 50, not nullable
     - **`Budget`**, `decimal`, precision 8, scale 2 not nullable
     - **`StartDate`**, `date`, not nullable
     -
2. Compile and run the application. Check the database.
   Hint: If there are no errors, but the 2 tables are not created, you'll need to add something to the DbContext.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 3: Add data

1. In the `Seed` method, keep the first line:

```
using var context  = new SchoolContext(options);
```

   Replace the remainder with the content of `Reference/seed-lab02.txt`.

»

Exercise 4: Show data

1. Show all courses in `CourseService`, method `AllCourses`:

```
1045 - Calculus, Credits = 4
...
```

2. Show all departments in `DepartmentService`, method `AllDepartments`:

```
1 - Engineering, Budget = 350000,00, Startdate = 01/09/2007
...
```

»

Exercise 5: One configuration class per entity

1. In the folder 'Dal' add a folder 'EntityConfigurations'.
2. Inside, create a configuration class for each of the three models. (Keep the `Person` configuration empty for now.)
3. Apply each configuration in the `OnModelCreating` event handler.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

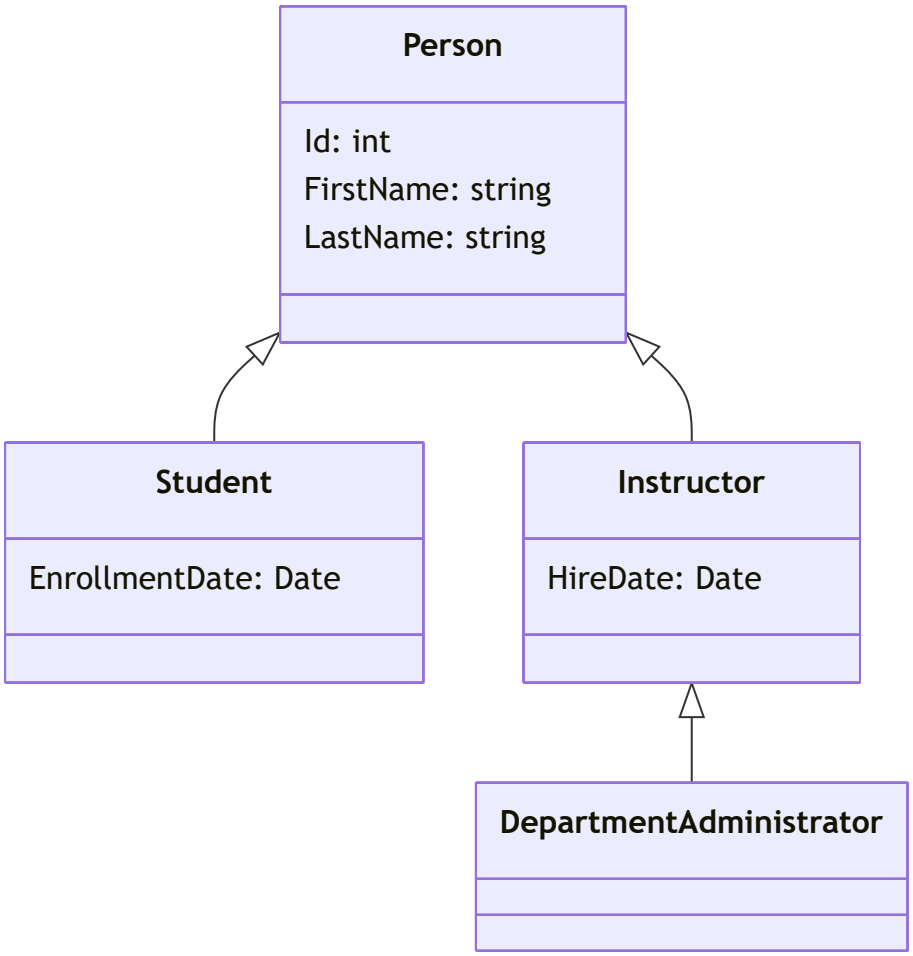
Lab 03 - Inheritance

Scenario

In this lab you'll add models to represent *inherited entities* and configuring them to map to new database tables.

Task 1 - Students, Instructors and DepartmentAdministrators

- Watch this diagram:



- A Student is a Person and has a property `EnrollmentDate` of type `Date`.
- An Instructor is a Person and has a property `HireDate` of type `Date`.
- A DepartmentAdministrator is an Instructor.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

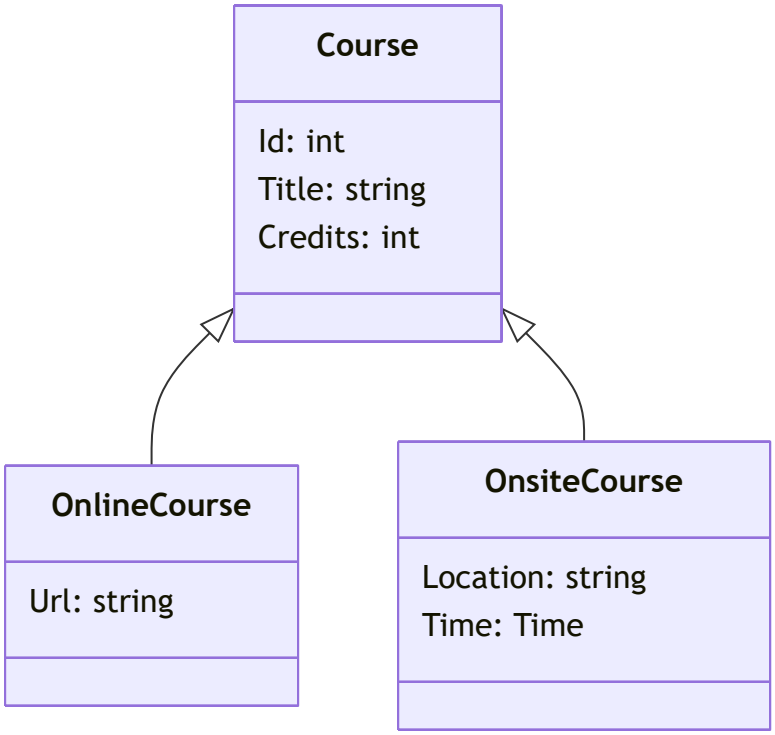
1. Add the new models to map this hierarchy to your application. There is no need to add model configurations yet. Update the `SchoolContext` accordingly. (Keep the `DbSet<Person>` property.)
2. All data should reside in one table: `People`.
   Before running the app, which database changes do you expect?
   Compile and run the app. Check the columns in the `People` table.
3. In the `Seed` method keep the first line and replace the remainder of the logic with the content of `Reference/seed-lab03.txt`. (Part of the code is commented out, this is on purpose.)
4. Build and run, no errors should occur.
5. Add logic to display the entities in the `PersonService`, methods:
   - `Students`
   - `Instructors`
   - `DepartmentAdministrators`

Note: Since an `Administrator` is an `Instructor`, when you show the `Instructors`, you may also see the `Administrators`. We will learn about queries and filters in a later lesson, so don't worry about that.

»

Task 2 - OnlineCourses and OnsiteCourses

- Watch this diagram:



- An `OnlineCourse` is a `Course` and has a `Url` of type `string`.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- An `OnsiteCourse` is a `Course` and has a `Location` of type `string` and a `Time` of type `Time`.

1. Like before, add the models to map this hierarchy to your application.
2. Update the `SchoolContext`.
3. This time, the database should get two additional tables: `OnlineCourses` and `OnsiteCourses`.
   Both of these tables will have their own `Id` of type `int`, not nullable, set as `Primary Key`. Their `Id` column is also set as `Foreign Key` to the table `Courses`.

   Add one line of code to enable this in `CourseConfiguration`.
4. In the `Seed` method remove the logic to add courses:

   ```
   List<Course> courses = [...];
   context.AddRange(courses);
   context.SaveChanges();
   ```

   Uncomment the logic to add onlineCourses and onsiteCourses.
5. Add logic to display the entities in the `CourseService`, methods:
   - `OnlineCourses`
   - `OnsiteCourses`

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 04 - One to Many Relationship

Scenario

Your task is to modify the models to establish a one-to-many relationship between `Department` and `DepartmentAdministrator` and display the departments with their associated `AdministratorId`.

Details:

- A `Department` may have zero or one `DepartmentAdministrator`.
- A `DepartmentAdministrator` may have zero or more `Departments`.

Exercise 1: Modify Models to setup the Relationship

If you already have an idea of how to build the relationship, feel free to start coding, otherwise read further.

1. Update the `Department` model. Include a nullable `AdministratorId` property and an `Administrator` property of type `DepartmentAdministrator`.
2. Update the `DepartmentAdministrator` model. Include a collection of `Departments`.

Exercise 2: Add Data

1. In the `Seed` method, keep the first line and replace the remainder with the content of `Reference/seed-lab04.txt`. The only new thing is that departments now have got an `AdministratorId`.

Exercise 3: Display Departments

1. Update the logic in `DepartmentService`, method `AllDepartments`. Show the `AdministratorId` as well.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 05 - Many To Many Relationship

Scenario

You will extend your application by configuring many-to-many relationships between `Course` and `Instructor`, and `Course` and `Student`. This lab focuses on setting up the correct relationships in your models and displaying the join entities.

Task 1 - Course-Instructor (implicit join entity)

- One `Course` can be taught by multiple `Instructors`.
- One `Instructor` can teach multiple `Courses`.

1. Create a join table with minimum effort. Just add navigation properties. Compile and run the program. Inspect the generated table.
2. Add some fluent api in the course configuration file, to ensure that the names are different:
    - CoursesInstructors
        - **`CourseId`**, `int`, not nullable
        - **`InstructorId`**, `int`, not nullable

    Compile and run the program. Inspect the generated table.

Task 02 - Course-Student (explicit join entity and payload)

- One `Course` may be followed by multiple `Students`.
- One `Student` may be enrolled in multiple `Courses`.
- Each student `Enrollment` also stores the `Grade` of type nullable `decimal(3,2)`.

The database should get an `Enrollments` table with the following columns:

- `CourseId` INT NOT NULL
- `StudentId` INT NOT NULL
- `Grade` DECIMAL(3,2) NULL

`CourseId` and `StudentId` together form the primary key. `CourseId` is a foreign key related to `Course.Id`, and `StudentId` is a foreign key related to `Student.Id`.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

1. Create an `Enrollment` entity and use fluent-api to configure the entity.
2. Add a property to the `DbContext`. Table name should be `Enrollments`.
3. Modify the two related entities. Compile and run the application. Inspect the new table.
4. In the `Seed` method, keep the first line and replace the remainder with the content of `Reference/seed-lab05.txt`.
5. Display the `StudentId` and `CourseId` of all enrollments in `EnrollmentService`, method `DisplayAllEnrollments`:

```
StudentId CourseId
6         2021
6         2030
7         2021
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 06 - Querying Data

Scenario

In the previous labs you've added tables and data to the database. Let's use LINQ to query data as a real data scientist.

Exercise 1: Filter and Sort Students

- Update the query in `PersonService`, method `Students`. Sort the students by their enrollment date, from the earliest to the latest:

```
Student                EnrollmentDate
Meredith Browning      01/09/2000
Alexandra Walker       01/09/2000
Damien Jai             01/09/2001
...
```

Exercise 2: Find the First Instructor Hired

- Add logic to `PersonService`, method `FirstInstructorHired`, to display the first instructor hired, based on hire date:

```
Kim Abercrombie (11/03/1995)
```

Exercise 3: Calculating Average Grades

- Add logic to `PersonService`, method `AverageGradePerStudent`, to display the average grade for each StudentId:

```
StudentId  Grade
6          3,75
7          3,50
...
```

Exercise 4: Finding the Lowest Grade per Course

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- Add logic to `CourseService`, method `LowestGradePerCourse`, to find the lowest grade for each CourseId:

```
CourseId  Lowest Grade
1045      1,50
1050      3,50
...
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 07 - Load Related Data

Scenario

You'll write queries to **eagerly load** related data from the database. Eager loading allows you to load related entities as part of the initial query, reducing the number of queries needed to retrieve the data.

Then you'll practice **explicit loading** of related data. This method allows you to load related entities on demand, which is useful for optimizing performance and controlling data retrieval.

Exercise 1: Update AllDepartments

- First ensure that instead of `AdministratorId`, the full name of the `DepartmentAdministrator` is shown:

```
Department name: Engineering, Budget: 350.000,00, Administrator: Kim Abercrombie
Department name: English, Budget: 120.000,00, Administrator: Candace Kapoor
...
```

- Then try to also display the `Title` of all courses pertaining to the `DepartmentAdministrator`:

```
Department name: Engineering, Budget: 350.000,00, Administrator: Kim Abercrombie
        - Chemistry
Department name: English, Budget: 120.000,00, Administrator: Candace Kapoor
...
```

Exercise 2: Update AverageGradePerStudent

- Show the student's full name instead of `StudentId`:

```
Student                 Grade
Gytis Barzdukas         3,75
Peggy Justice           3,50
...
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 3: Update LowestGradePerCourse

- Show the course's `Title` instead of `CourseId`:

```
Calculus: 1,50
Chemistry: 3,50
```

Exercise 4: Explicitly Load Related Data

Prompt the user to enter the `Id` of the `Instructor` they want to see more information about. Use the private static method `GetInstructorId`.

- Explicitly load the courses related to the selected `Instructor`. If the Instructor cannot be found show a message:

```
No instructor found with id 66.
```

- Display the title of the courses that the `Instructor` is teaching. If there are no courses, display "No courses assigned":

```
Enter instructor id: 31
Instructor Kim Abercrombie
Teaching:
        - Chemistry
```

```
Enter instructor id: 34
Instructor Candace Kapoor
Teaching:
        - No courses assigned
```

Hint: Use the `Entry` method of the `DbContext` to access the `Collection` method for explicit loading.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 08 - Pagination

⭐ This lab is optional ⭐

Scenario

You're going to display a list of people from the database. The application will allow users to navigate through the list in pages of 5 people each, using **keyset pagination** for efficient data retrieval.

Exercise 1: Pagination Logic

- Comment out the method `AllPeople`. Add a new `AllPeople` above it, along with a `Previous` and `Next` method:

```
public List<Person> Previous(int firstPersonId, int pageSize)
{
     // TODO
}

public List<Person> Next(int lastPersonId, int pageSize)
{
    // TODO
}
```

- In `AllPeople` initialize pagination variables (`pageSize`, `firstPersonId`, `lastPersonId`).
- Fetch the first page of people using the `Next` method.

Exercise 2: User Interaction

- Display the list of people in the current page.
- Prompt the user to navigate:

```
1 - Fadi Fakhouri
2 - Stacy Serrano
3 - Jasmine Stewart
4 - Kristen Xu
5 - Roger Van Houten
(P)revious  (N)ext  (Q)uit
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 3: Implement `Previous`

- Retrieve people whose `Id` values are less than the `Id` of the first person in the current page.
- Make sure the results are ordered in ascending order by `Id` so the page displays correctly.
- Hint: You may need to order by descending `Id`, take the desired number of results, and then re-order ascending before returning.

Exercise 4: Implement `Next`

- Retrieve people whose `Id` values are greater than the `Id` of the last person in the current page.
- Return the next set of people, ordered by ascending `Id`.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 09 - Basic SaveChanges

Scenario

You will focus on the `SaveChanges` method to perform basic data operations such as inserting, updating, and deleting a single `Department` entity.

In `DepartmentService` lives the method `AddUpdateDeleteDepartment`. This method calls 3 private methods which you'll implement in the next exercises.

Exercise 1: Add a New Department

- Prompt the user to enter the name and budget for a new department. Copy/paste next code in `AddNewDepartment`:

```
Console.Write("Department Name: ");
var name = Console.ReadLine();
if (name == null)
{
    Console.WriteLine($"Not a valid name: {name}");
    return;
}

Console.Write("Department Budget: ");
var input = Console.ReadLine();
if (!decimal.TryParse(input, out var budget))
{
    Console.WriteLine($"Not a valid budget: {input}");
    return;
}
```

- Set the start date to the current date.
- Insert the new department into the database and save the changes.
- Show a confirmation message afterwards:

```
Department Name: Test
Department Budget: 100000
Department 'Test' added with ID 5
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 2: Update the Budget of the Most Recently Added Department

- Identify the most recently added department.
- Prompt the user to enter a new budget:

```
New budget for department 'Test': 130000
```

- Update the budget of the identified department and save the changes.
- Show a confirmation message:

```
Department 'Test' has got a new budget: 130000
```

»

Exercise 3: Delete the Most Recently Added Department

- Identify the most recently added department.
- Delete the identified department from the database and save the changes.
- Show a confirmation message:

```
Department 'Test' has been deleted
```

»

Exercise 4: Show State Entity

1. Review the method `AddUpdateDeleteDepartment` (`DepartmentService`). Look at the code you added earlier.
2. In the method `AddNewDepartment` add a statements before and after `SaveChanges`:

```
Console.WriteLine($"***** State: {context.Entry(department).State} BEFORE
SaveChanges *****");
context.SaveChanges();
Console.WriteLine($"***** State: {context.Entry(department).State} AFTER
SaveChanges *****");
```

3. Try to predict what will be shown and run the logic.
4. Repeat above steps for the `UpdateBudgetLastAddedDepartment`
5. Repeat above steps for the `DeleteLastAddedDepartment`

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 5: Detect Changes and use ChangeTracker.DebugView

1. Remove the lines from previous exercise.
2. Add a helper in `DepartmentService`:

```
private static void DisplayChangeTrackerDebugView(SchoolContext context, bool
detectChanges = true)
{
    var defaultColor = Console.ForegroundColor;

    if (detectChanges)
    {
        context.ChangeTracker.DetectChanges();
    }
    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine(context.ChangeTracker.DebugView.LongView);
    Console.ForegroundColor = defaultColor;
}
```

3. Call this helper just before the `SaveChanges` in `UpdateBudgetLastAddedDepartment`:

```
DisplayChangeTrackerDebugView(context);
```

   Test.

4. Also call this helper just after the `SaveChanges`. Test.
5. Update the first call to not detect changes:

```
DisplayChangeTrackerDebugView(context, false);
```

   Test. Remove the second argument (`false`).

»

Exercise 6: PropertyEntry

1. Add another helper:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
private void DisplayPropertyEntries(IEnumerable<PropertyEntry> propertyEntries)
{
    var defaultColor = Console.ForegroundColor;
    foreach (PropertyEntry property in propertyEntries)
    {
        Console.ForegroundColor = ConsoleColor.Magenta;

        var propertyName = property.Metadata.Name;
        var currentValue = property.CurrentValue;
        var originalValue = property.OriginalValue;
        Console.WriteLine($"{propertyName}: CurrentValue={currentValue},
OriginalValue={originalValue}");

        Console.ForegroundColor = defaultColor;
    }
}
```

2. Call it in `UpdateBudgetLastAddedDepartment`:

```
DisplayChangeTrackerDebugView(context);
DisplayPropertyEntries(context.Entry(department).Properties);   // 👈

context.SaveChanges();

DisplayChangeTrackerDebugView(context);
DisplayPropertyEntries(context.Entry(department).Properties);   // 👈
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 10 - Delete Related Data

Scenario

You'll extend the database schema with a one-to-many relationhip between `Department` and `Course`. In the configuration ensure that departments with related courses cannot be deleted.

Then you'll add logic to:

- delete a Department Administrator and keep associated departments;
- delete a Department Administrator with associated departments (*unless* the departments have associated courses, then an exception will be thrown which needs to be handled).

Exercise 1: Create one-to-many between Department and Course

1. Update both models for this relationship:
    - a department can have zero or more courses
    - a course *must* have a `DepartmentId`
2. Add fluent api for `Course` to ensure that when the user tries to delete a `Department` having related courses, the database raises an error.
3. In the `Seed` method, keep the first line and replace the remainder with the content of `Reference/seed-lab10.txt`. Both onlineCourses and onsiteCourses now have got a `DepartmentId`.
4. Build and run the application. No errors should occur.

Exercise 2: Update the relationship Department - Administrator

1. This relationship is already optional as the `AdministratorId` foreign key in the `Department` entity is `nullable`.
2. Add fluent api to ensure that when an `Administrator` is deleted, the `AdministratorId` of all related departments is set to `null` *directly* in the database.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Exercise 3: Implement

DeleteDepartmentAdministratorKeepDepartments

1. Allow the user to delete an administrator without deleting their departments. Start the method with:

```
Console.Write("Enter administrator id: ");
var administratorId = int.Parse(Console.ReadLine()!);
```

   Note. Kim Abercrombie (Id 31) is the administrator of department 'Engineering'. After deleting Kim, check Departments, Display All Departments. 'Engineering' should exist *without* administrator.

2. After deletion show a confirmation:

```
Enter administrator id: 34
Department Administrator Candace Kapoor deleted and 1 related department(s) were
kept
```

Exercise 4: Implement

DeleteDepartmentAdministratorWithDepartments

(if time permits)

1. In the `Seed` method, temporarily update the `DepartmentId` of the courses "Trigonometry" and "Calculus" from 4 to 1.
2. Allow the user to delete an administrator along with their departments. Ensure the application catches database errors when trying to delete a department that has related courses.
3. Alway inform the user appropriately:

```
Enter administrator id: 32
Department Administrator Roger Harui and 1 related department(s) deleted
```

```
Enter administrator id: 34
Error deleting the Department Administrator. They may have courses associated with
it.
The DELETE statement conflicted with the REFERENCE constraint
"FK_Courses_Departments_DepartmentId". The conflict occurred in database
"SchoolV2", table "dbo.Courses", column 'DepartmentId'.
```

4. Undo the change in the `Seed` method.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 11 - Manage Concurrency Conflicts

Scenario

You will extend the application to manage concurrency. The goal is to ensure that updates to the `Departments` table handle concurrency appropriately.

Exercise 1: Entity Configuration

1. The **Departments Table** has a `Name` column of type `varchar(50)`, which is not nullable. Configure the `Department` entity to use the `Name` column as a concurrency token during updates.

Exercise 2: Implement Update Operation

1. In `DepartmentService` implement the method `UpdateDepartment`. Start with the happy path (no concurrency conflicts).
   As a starting point copy/paste next logic (don't worry about the errors):

```
Console.Write("Enter Department Id: ");
var departmentId = int.Parse(Console.ReadLine()!);

Console.WriteLine($"Id = {department.Id}, Name = {department.Name}, Budget =
{department.Budget}, Startdate = {department.StartDate}, AdministratorId =
{department.AdministratorId}");

Console.Write("Enter the new name: ");
var newName = Console.ReadLine()!;

Console.Write("Enter the new budget: ");
var newBudget = decimal.Parse(Console.ReadLine()!);

Console.Write("Enter the new start date: ");
var newStartDate = DateOnly.Parse(Console.ReadLine()!);
```

2. Implement the remainder of the logic. Update a department with the given user input, save to the database and show a confirmation message afterwards.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
Enter Department Id: 1
Id = 1, Name = Engineering, Budget = 350000,00, Startdate = 01/09/2007,
AdministratorId = 31
Enter the new name: Software Development
Enter the new budget: 999999
Enter the new start date: 25/5/2026
Department updated:
Id = 1, Name = Software Development, Budget = 999999, Startdate = 25/05/2026,
AdministratorId = 31
```



Exercise 3: Add Concurrent Action

1. Before the `SaveChanges()` add a line of code to mimick another user touching the same data.

```
   // Simulate a change in the database by another user
   context.Database.ExecuteSql($"Update Departments SET Name = 'Something
different' WHERE Id = {departmentId}");
```

2. Test. An exception will be thrown:

```
Microsoft.EntityFrameworkCore.DbUpdateConcurrencyException: 'The database
operation was expected to affect 1 row(s), but actually affected 0 row(s); data
may have been modified or deleted since entities were loaded.
```



Exercise 4: Exception Handling

1. Handle this exception to manage conflicts and prompt the user for resolution.
2. Implement and test the 🗄️ **Database Wins** strategy.
3. Implement and test the 👷 **Client Wins** strategy.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 12 - ExecuteUpdate

Scenario

You will implement a feature to increase the credits of all courses in a selected department. Utilize the `ExecuteUpdate` method to efficiently update records in the database.

Exercise 1: Implement Increase Credits Of Courses In Selected Department

1. In `CourseService`, method `IncreaseCreditsOfCoursesInSelectedDepartment`, prompt the user to enter the department ID:

```
Console.Write("Provide Department Id: ");
var departmentId = int.Parse(Console.ReadLine()!);
```

2. Implement functionality to increase the credits of all courses within that department by 10.
3. Use the `ExecuteUpdate` method to perform the update efficiently.
4. Show a confirmation message afterwards:

```
Credits increased for 2 courses for department 1
```

5. If either the department doesn't exist or the department has no courses, show a message as well:

```
No Courses found for department with id 5
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 13 - SQL Queries

Scenario

Execute SQL queries using `FromSql`, `SqlQuery`, and stored procedures in EF Core. The focus is on querying the database to retrieve and display specific information.

Exercise 1: Show Best 5 Students

1. Write a method in `PersonService`, method `Best5Students`, to execute the following SQL query and display the results:

```
SELECT TOP 5 p.Id, p.FirstName, p.LastName, p.Discriminator, p.HireDate,
p.EnrollmentDate
FROM People p
INNER JOIN Enrollments e ON p.Id = e.StudentId
WHERE p.Discriminator = 'Student'
GROUP BY p.Id, p.FirstName, p.LastName, p.Discriminator, p.HireDate,
p.EnrollmentDate
HAVING AVG(e.Grade) IS NOT NULL
ORDER BY AVG(e.Grade) DESC
```

2. Use `FromSql` or `FromSqlRaw` to execute this query against the `Students` DbSet.
3. Display the ID, first name, and last name of the top 5 students:

```
15 - Arturo Anand
21 - Robyn Suarez
...
```

Exercise 2: Stored Procedure

1. Add logic to the `Seed` method to create this stored procedure:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
CREATE PROCEDURE dbo.GetDepartmentsWithEnrollmentsAbove
@NumberOfEnrollments INT
AS
BEGIN
   SELECT d.Id, d.Name, d.StartDate, d.Budget, d.AdministratorId
   FROM Departments d
   INNER JOIN Courses c ON d.Id = c.DepartmentId
   INNER JOIN Enrollments e ON c.Id = e.CourseId
   GROUP BY d.Id, d.Name, d.StartDate, d.Budget, d.AdministratorId
   HAVING COUNT(e.CourseId) > @NumberOfEnrollments
   ORDER BY d.Name DESC
END;
```

2. Add logic in `DepartmentService`, method `GetDepartmentsWithEnrollmentsAbove` to invoke the

   stored procedure. The stored procedure accepts a parameter `NumberOfEnrollments` and returns departments with more than the specified number of enrollments.
3. Use `FromSql` or `FromSqlRaw` to execute the stored procedure with a parameter value of `5`.
4. Display the results:

```
Show Departments with more than 5 enrollments:
  - Economics
  - Engineering
  - English
```

»

Exercise 3: Show Students With Courses

(if time permits)

1. In a new folder 'ViewModels' create a class `StudentWithCourse`:

```
public class StudentWithCourse
{
    public string FirstName { get; set; } = null!;
    public string LastName { get; set; } = null!;
    public string Title { get; set; } = null!;
}
```

2. In the `EnrollmentService`, method `DisplayStudentsAndCourses`, add logic to execute the following SQL query and map the results to the `StudentWithCourse` ViewModel:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
SELECT
p.FirstName, p.LastName, c.Title
FROM Enrollments AS e
INNER JOIN Courses AS c
ON e.CourseId = c.Id
INNER JOIN People AS p
ON e.StudentId = p.Id
ORDER BY p.FirstName, c.Title
```

3. Use `SqlQuery` or `SqlQueryRaw` to execute the query.
4. Display the results:

```
Student: Alexandra Walker - Course: Macroeconomics
Student: Alexandra Walker - Course: Microeconomics
...
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 14 - Using DB Functions

⭐ This lab is optional ⭐

Scenario

You will be leveraging EF Core Functions that map to

SQL Server. Specifically, you will use `EF.Functions.StandardDeviationPopulation` to calculate the standard deviation of grades for each course.

Exercise 1 - StandDeviationPopulation

Standard deviation shows how much values in a data set typically differ from the mean. A small standard deviation means most values are close to the average; a large one means the values are more spread out.

In this lab you'll compute the population standard deviation of grades for each course using `EF.Functions.StandardDeviationPopulation` so you can see how consistent (or variable) grading is within a course. If you are interested in the algorithm behind, see example below:

```
- Grades: 80, 90
- Mean = (80 + 90) / 2 = 85
- Variance (population) = ((80-85)^2 + (90-85)^2) / 2 = (25 + 25) / 2 = 25
- Standard deviation = sqrt(25) = 5.0000
```

1. In `CourseService`, method `CourseGradeStandardDeviations`, implement the logic that calculates the standard deviation of grades for each course.

   The method should:

   - Use the `Enrollments` table to group grades by `CourseId`.
   - Use `EF.Functions.StandardDeviationPopulation` to calculate the standard deviation of grades within each group.
   - Filter out grades which are `null`.
2. The method should display the `CourseId` and the calculated standard deviation of grades for each course.
3. Format the standard deviation to four decimal places for better readability.

   ```
   CourseId: 1045 => 0,5000
   CourseId: 1050 => 0,0000
   ...
   ```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 15 - Basic Migrations

Scenario

You'll split up the current project into two projects: `School.ConsoleApp` and `School.Library`. This will make the solution a bit more 'real-world'. The DbContext must be updated to support migrations. Finally you will use EF Core Migrations to generate a brand new database with the same schema that we've been using so far. Don't worry about filling the database with data, yet.

Exercise 1: Add Library project

1. Update all installed NuGet packages to the latest version.
2. Add a Class Library project `School.Library` to the solution.
3. Add a project reference in the ConsoleApp to the Library.
4. Move the folders `DAL`, `Models`, `ViewModels` and `Services` to the Library. Keep the folder `Menu` and `appsetttings.json` in the ConsoleApp project.
5. Remove the package `Microsoft.EntityFrameworkCore.SqlServer` from the ConsoleApp project, and add it to the Library project.
6. Update namespaces accordingly.
7. Test if the ConsoleApp still runs.

Exercise 2: Comment out creation and fill logic

1. Comment out the 'quick prototyping' logic which is responsible for dropping and creating the database with each run:

```
context.Database.EnsureDeleted();
context.Database.EnsureCreated();
```

2. Also comment out the call to the `SchoolInitializer.Seed` method.
3. Test if the ConsoleApp still runs.

Exercise 3: Update Connection String

1. Depending on your setup, you might have a connection string in the:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- `Program.cs`, or
- `appSettings.json` Modify the connection string to point to a new database: SchoolV3.

»

Exercise 4: Update DbContext

1. Add a parameterless constructor to the `SchoolContext`.
2. Override the `OnConfiguring method`. Details in the code:

```
// Constructor via which SchoolApp provide the connection details
public SchoolContext(DbContextOptions<SchoolContext> contextOptions):
base(contextOptions)
{
}

// 👇 Add parameterless constructor to support migrations
public SchoolContext()
{
}

// 👇 Override OnConfiguring to support migrations
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    // 👇 The program wasn't started by the console app apparently
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseSqlServer(/* connection string here */);
    }
}
```

3. Copy the correct connection string into the `OnConfiguring` method. We don't like hard-coded connection strings, but we keep it as is for now.

»

Exercise 5: Install EF Tools and Design

1. As `School.ConsoleApp` is the startup project, add the package `Microsoft.EntityFrameworkCore.Design` to the `School.ConsoleApp` project.
2. As `School.Library` contains the DbContext, add the package `Microsoft.EntityFrameworkCore.Tools` to the `School.Library` project.

»

Exercise 6: Add Migration

1. Open the Package Manager Console.
2. Select School.Library as default project and run:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
Add-Migration InitialCreate
```

- Inspect the generated Migrations folder.

Exercise 7: Build Database

1. Build the new database with all tables, fields and relations:

```
Update-Database
```

2. Check if the database SchoolV3 has been created and the ConsoleApp runs (without any data yet).

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 16 - Customize Migration

In an earlier lab we worked with a stored procedure. You might have noticed that the migration we ran created a SchoolV3 database with the correct table schema but did not include any stored procedure. In this lab, you'll address this by manually creating a new migration and adding the SQL code to create the missing element.

Exercise 1: Add an Empty Migration

1. Add an empty migration to your project class library. In PMC select `School.Library' as default project and run:

```
Add-Migration SqlCustomization
```

Exercise 2: Add DB Elements to the Migration

1. Find the stored procedure `GetDepartmentsWithEnrollmentsAbove` in the `Seed` method of the `SchoolInitializer` class.
2. Modify the `Up` method of the new migration to execute the SQL statement to create the stored procedure. Use the `migrationBuilder.Sql` method.

```
migrationBuilder.Sql(@"
    CREATE PROCEDURE [dbo].[YourStoredProcedure]
    AS
    BEGIN
        -- Your SQL code here
    END
");
```

Exercise 4: Drop the Items in the Down Method

1. In the `Down` method of the new migration, add the `migrationBuilder.Sql` statements to drop the Store Procedure.

```
migrationBuilder.Sql("DROP PROCEDURE [dbo].[YourStoredProcedure]");
```

Exercise 5: Apply the Migration

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- Execute the `Update-Database` command to apply the migration.
- Verify that the stored procedure has been added to the SchoolV3 database.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



Lab 17 - Seed Data

Your new SchoolV3 database is currently empty. It contains tables, keys, indexes and a stored procedure, but it lacks data. Your task is to write code to insert data into these tables, ensuring that the data gets inserted during a migration.



Exercise 1: Insert Data Using the HasData Method

In the entity configuration classes use the `HasData` method to add the seed data.

1. Add the content of `Reference/seed-lab-17-Department` to your `DepartmentConfiguration`.
2. Add the content of `Reference/seed-lab-17-Enrollment` to your `EnrollmentConfiguration`.
3. Add all configuration files from `Reference/Configurations` to `DAL/EntityConfigurations` *and* Apply them:

```
modelBuilder.ApplyConfiguration(new DepartmentAdministratorConfiguration());
modelBuilder.ApplyConfiguration(new InstructorConfiguration());
modelBuilder.ApplyConfiguration(new OnlineCourseConfiguration());
modelBuilder.ApplyConfiguration(new OnsiteCourseConfiguration());
modelBuilder.ApplyConfiguration(new StudentConfiguration());
```



Exercise 2: Add the Migration

1. In the PMC select the Class Library project as default project.
2. Add a migration to generate the seed data by running the following command:

```
Add-Migration SeedData
```



Exercise 3: Customize the `Up` Method

Since we don't have a class to map the table **`CoursesInstructors`**, we can't use the `HasData` method for that. This means that you will have to customize the new migration to add the rows manually.

1. Add the following items in the `Up` method:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
migrationBuilder.InsertData(
    table: "CoursesInstructors",
    columns: new[] { "CourseId", "InstructorId" },
    values: new object[,]
    {
        {2030, 1},
        {2021, 2},
        {1061, 3},
        {4041, 4},
        {4061, 5},
        {1050, 31},
        {1045, 32},
        {4022, 33},
        {2042, 34}
    });
```

Exercise 4: Customize the `Down` Method

1. Ensure the `Down` method of the new migration class deletes the `CoursesInstructors` items *before* the courses and instructors are deleted. This is important to avoid foreign key constraint violations when rolling back the migration.

```
migrationBuilder.DeleteData(
    table: "CoursesInstructors",
    keyColumns: new[] { "CourseId", "InstructorId" },
    keyValues: new object[,]
    {
        { 2030, 1 },
        { 2021, 2 },
        { 1061, 3 },
        { 4041, 4 },
        { 4061, 5 },
        { 1050, 31 },
        { 1045, 32 },
        { 4022, 33 },
        { 2042, 34 }
    });
```

Exercise 5: Apply the Migration

1. Execute the `Update-Database` command to apply the migration.
2. Build and run the app. All should work fine again!

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 18 - DBA Script

⭐ This lab is optional ⭐

You'll have to deploy your database to another machine using EF Core's migration tools. Let's build a SQL script, which is useful for DBAs to review before execution.

Prerequisites

Ensure that you have completed the previous labs and have the following setup:

- Multiple projects: Console Application and Class Library.
- Existing migrations set up and ready to go.

Exercise 1: Generate SQL Scripts

- In the PMC select the Class Library project as default project.
- Generate the SQL scripts for all the migrations by executing:

```
Script-Migration -Output "sql-scripts\script.sql"
```

- This command will create a SQL script that includes all the necessary commands to create and seed the database.

Exercise 2: Review the SQL Scripts

- Open the generated SQL script in a text editor.
- Review the script to ensure it contains the expected commands for creating tables, constraints, indexes, and seeding data.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 20 - Logging

⭐ This lab is optional ⭐

Scenario

You are going to enable simple logging on your existing application.

Exercise 1: ChangeTracking and LogLevel.Information

1. Configure your Console Application to log the `Microsoft.EntityFrameworkCore.ChangeTracking` category on the console.
2. Filter the log level to `LogLevel.Information`.
3. Run the application and execute `AddUpdateDeleteDepartment`.
4. Inspect the console and notice no logs.

Exercise 2: ChangeTracking and LogLevel.Debug

1. Filter the log level to `LogLevel.Debug`.
2. Run the application and execute `AddUpdateDeleteDepartment`.
3. Inspect the console and notice the logs, missing the Id of the tracked entity.

Exercise 3: Sensitive Data Logging

1. Enable Sensitive Data Logging.
2. Run the application and execute `AddUpdateDeleteDepartment`.
3. Inspect the console and notice the logs, containing the Id of the tracked entity.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support